

Table of Contents

PEPG 3: A Detailed System Architecture and Operational Specification of the Participatory Governance Framework

Milad Habibpour

ORCID: 0009-0001-4789-0333

Conceptual Architecture and Operational Specification Paper

Updated Edition - September 2026

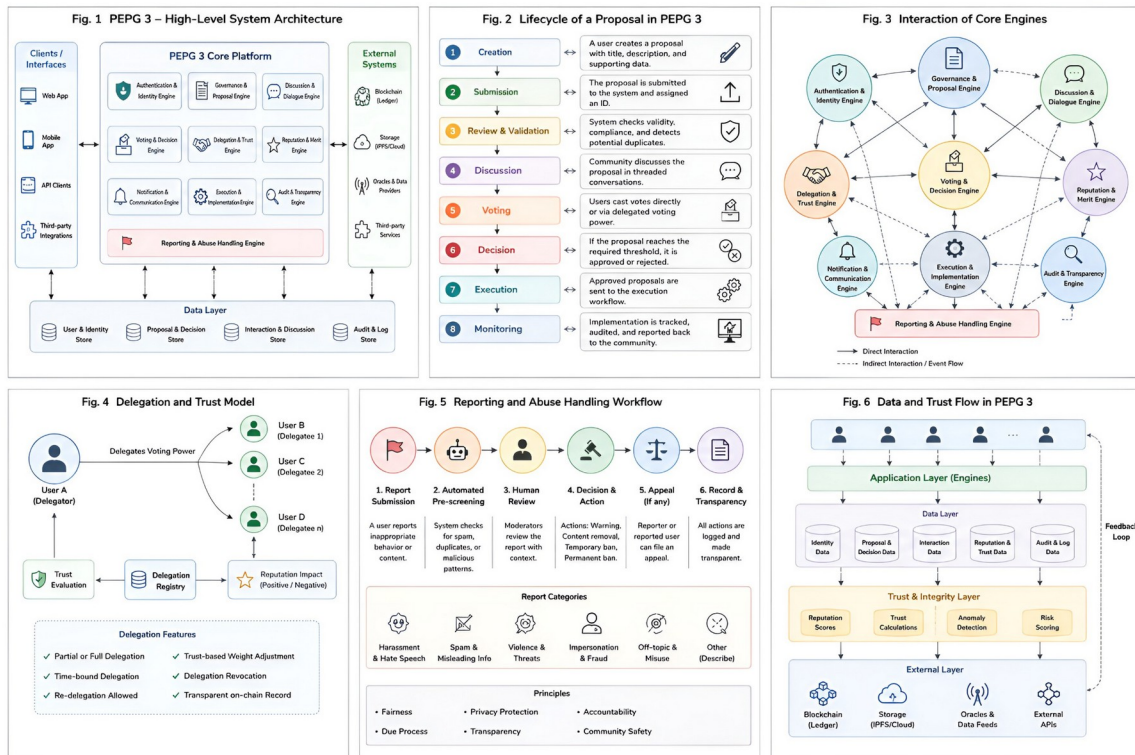


Figure plate. The six supplied PEPG 3 architecture views.

Abstract

Contemporary digital platforms have substantially increased the capacity of individuals to communicate, coordinate, deliberate, and participate in collective decision-making. Nevertheless, communication, voting, representation, project execution, identity verification, and accountability are commonly implemented as separate functions. This fragmentation creates a fundamental institutional gap: a platform may enable users to express preferences without establishing whether participants are trustworthy, whether their claims are verifiable, whether representation is contextually appropriate, whether discussions produce structured outcomes, or whether collectively adopted decisions can be translated into transparent and accountable execution.

This paper proposes PEPG as a conceptual architecture for a problem-oriented digital governance system that integrates identity and trust verification, contextual representation, liquid delegation, structured deliberation, collective decision-making, project governance, monitoring, verification, and closure within a unified lifecycle. The architecture treats the problem, rather than the institution or the vote itself, as the central unit of governance.

PEPG introduces several interconnected governance engines. The Trust and Verification Engine establishes verifiable profiles and contextual confidence scores for identity, education, professional expertise, location, activity, and other claims. The Delegation Engine allows users to transfer voting rights to other participants at multiple scopes, including an entire problem, a particular discussion, a defined domain, or a broader governance scope. The Discussion and Deliberation Engine transforms unstructured communication into structured discussions, proposals, evidence, highlights, sub-discussions, outputs, and decisions. The Voting Engine resolves direct and delegated voting power within the relevant scope. The Execution and Project Governance Engine transforms approved practical solutions into projects and sub-projects governed through managers, teams, supervisors, suppliers, requirements, budgets, schedules, contracts, progress reports, challenges, and monitoring. Finally, the Accountability and Trust Layer provides reporting, verification, dispute handling, and reputation feedback.

The proposed architecture therefore conceptualizes governance as a continuous cycle:

Problem → Context → Trust → Deliberation → Representation → Decision → Execution → Verification → Evaluation → Closure → Learning.

The paper argues that such integration can provide a foundation for a new class of digital governance systems in which participation is not limited to voting, representation is contextual rather than fixed, trust is evidence-based rather than binary, deliberation produces structured outputs, and execution remains part of the governance process rather than occurring outside it.

Keywords: participatory governance, digital governance, liquid democracy, delegated voting, deliberative democracy, contextual representation, trust systems, collective intelligence, project governance, platform governance, decentralized governance, accountability.

1. Introduction

Digital communication technologies have transformed the scale and speed at which individuals can participate in collective processes. Yet most digital participation systems remain functionally fragmented. A platform may provide identity management, another may provide discussion, another voting, another project

management, and another reputation or moderation. These components rarely form a coherent governance architecture.

This fragmentation produces a fundamental problem.

A collective decision cannot be understood only by asking:

Who voted?

A more complete governance system must also ask:

- Who is the participant?
- Which claims made by that participant are verified?
- What is the participant's relevant expertise?
- In what context is that expertise relevant?
- Who has delegated authority to the participant?
- What discussion generated the decision?
- What evidence informed the discussion?
- What alternatives were considered?
- How was the final decision reached?
- Who is responsible for execution?
- How is execution monitored?
- Who verifies the result?
- How is the final outcome evaluated?

PEPG is proposed as an architectural response to this problem.

The central proposition is that **governance should be organized around problems and their complete lifecycle rather than around isolated voting events.**

Under this model, a problem is not simply an item submitted for voting. It becomes a persistent governance object containing its participants, contextual classification, discussions, evidence, proposals, decisions, projects, contracts, reports, verification records, and final evaluation.

This approach also extends ideas associated with Liquid Democracy. The literature commonly characterizes Liquid Democracy through voluntary delegation, proxy voting, and a flexible combination of direct and representative participation. Participants may delegate different issues to different representatives, and delegations may be transitively transferred.

PEPG adopts the principle of flexible delegation but embeds it inside a broader contextual and trust-oriented architecture.

Thus, PEPG is not merely a Liquid Democracy implementation.

It is a proposed **problem-oriented governance architecture** in which delegation is only one component of a larger system.

2. The Central Concept: Problem-Oriented Governance

The fundamental object in PEPG is the **Problem**.

A problem may represent:

- a social issue,
- a municipal problem,
- an organizational challenge,
- an infrastructure failure,
- an environmental concern,
- a scientific question,
- a community dispute,
- a development requirement,
- or any other collective matter requiring deliberation and potentially collective action.

A problem possesses a lifecycle.

Problem Creation



Context Identification



Participant Formation



Discussion



Solution Formation



Collective Decision



Execution Decision



Project Formation



Implementation



Monitoring



Verification



Evaluation



Closure



Learning

The lifecycle is not necessarily strictly linear.

A project may generate a new problem.

A discussion may generate a new discussion.

Execution may generate a new requirement.

A verification process may identify a failure.

A closed problem may create knowledge that improves future decision-making.

Therefore, PEPG is better modeled as a **dynamic governance graph** than as a simple workflow.

3. Architectural Principles

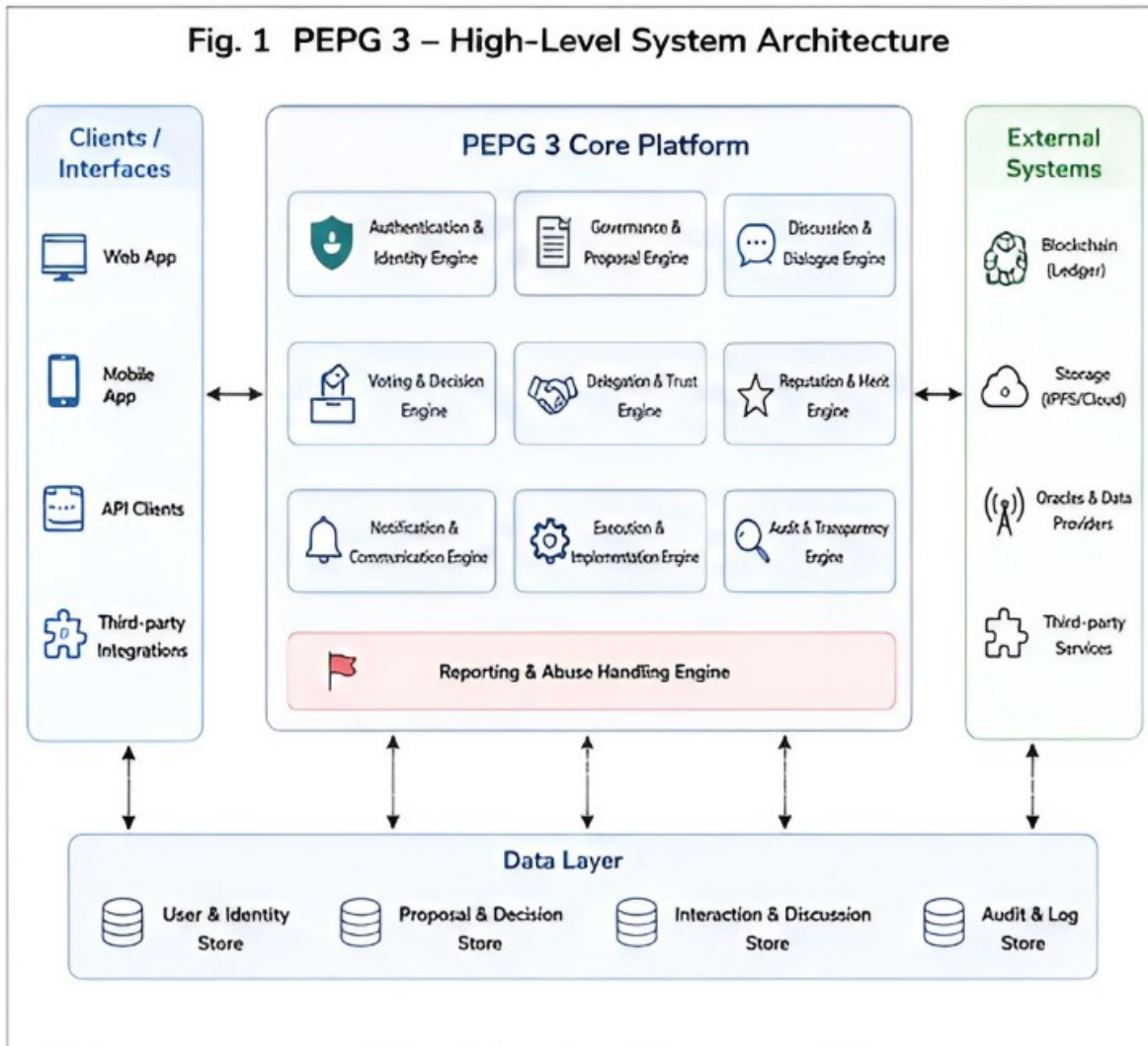


Figure 1. PEPG 3 high-level system architecture.

The architecture is based on several principles.

3.1 Contextuality

Trust, expertise, representation, and decision-making should be interpreted within a relevant context.

A person may be highly trustworthy with respect to one domain and have little demonstrated expertise in another.

Therefore:

Trust is contextual.

Similarly:

Representation is contextual.

And:

Decision-making is contextual.

3.2 Verifiability

Claims made by participants should not automatically be treated as equally reliable.

The system therefore distinguishes between:

- self-declared information,
 - community-confirmed information,
 - institutionally verified information,
 - and evidence-supported information.
-

3.3 Voluntary Representation

Users retain the ability to participate directly or delegate their voting authority.

This reflects the core logic of Liquid Democracy, which combines direct participation with voluntary proxy delegation.

3.4 Deliberation Before Decision

Voting should not necessarily be the first step.

PEPG places structured discussion before collective decision-making.

The objective is to allow participants to:

- exchange information,
 - challenge claims,
 - propose alternatives,
 - provide evidence,
 - formulate sub-questions,
 - identify disagreements,
 - and develop possible solutions.
-

3.5 Execution as Governance

A collective decision has limited institutional value if there is no transparent mechanism for implementing it.

PEPG therefore incorporates execution into the governance architecture.

The project is not an external administrative process.

It is a continuation of the governance lifecycle.

3.6 Accountability

Every important governance action should be attributable, auditable, and contestable.

This includes:

- profile verification,
 - delegation,
 - voting,
 - proposals,
 - reports,
 - project updates,
 - supervision,
 - contractual commitments,
 - and closure.
-

4. The Trust and Verification Engine

The Trust and Verification Engine is the foundation of PEPG.

Before participating in meaningful governance processes, a user creates a profile.

The profile may contain:

Identity

- name,
- family name,
- date of birth,
- relevant identifiers.

Education

- degree,
- field,
- institution,
- qualifications.

Professional

Profile

- profession,
- skills,
- experience,
- current activities.

Location

- residence,
- geographic area of activity,
- areas in which the user can provide services or participate.

Current

Activities

- projects,
- professional activities,
- organizational affiliations,
- areas of engagement.

The crucial architectural principle is that these attributes are not merely stored.

They are **verifiable claims**.

4.1 Field-Level Verification

Instead of assigning a single global trust score to a person, PEPG evaluates individual claims.

For example:

Name

100% verified

Education

94% verified

Civil Engineering

96% verified

Current Professional Activity

82% verified

Activity Location

87% verified

This avoids the conceptual problem of reducing a multidimensional person to a single number.

4.2 Verification Sources

A claim may be verified through several sources.

Institutional

Verification

For example:

- civil registry,
- university,
- professional association,
- government institution,
- licensed organization.

Institutional confirmation may establish a claim as officially verified.

Community

Verification

A user may nominate up to five other users to verify a particular claim.

For example:

“Do you confirm that this user is a Civil Engineer?”

The nominated users receive a request.

Each may respond:

- Confirm,
- Reject,
- Unable to verify.

The Verification Engine aggregates the responses according to the platform’s defined verification model.

4.3 Verification Transparency

The profile does not merely display:

100%

It should allow the user to inspect the basis of the result.

For example:

Name

100% Verified

Verification Sources:

- ✓ Official Registry
- ✓ Community Confirmation
- ✓ Identity Document

This creates **explainable trust** rather than opaque reputation.

5. Trust, Reputation and Expertise

PEPG must distinguish three concepts:

Identity



Who is this person?

Trust / Verification



Are the person's claims supported?

Contextual Reputation



How reliable has this person been
in this particular domain or activity?

A user may therefore have:

Identity Verification: 100%

Education Verification: 100%

Civil Engineering Expertise: 95%

Project Management Reputation: 88%

Environmental Governance Reputation: 63%

This contextual model is particularly important because research on delegated voting indicates that delegation can create both benefits and risks, including concentration of influence and reduced diversity of independent information.

PEPG therefore should not treat trust as a universal scalar.

6. User Reporting and Accountability

The Trust Engine must also contain a **User Reporting and Accountability Mechanism**.

Any user may report another user.

However, reporting must not automatically mean punishment.

A report is an **accountability claim**, not a verdict.

For example, a user may report another participant for:

- false information,
- impersonation,
- harassment,
- abusive behavior,
- manipulation,
- fraudulent claims,
- spam,
- coordinated disruption,
- inappropriate content,
- or misuse of delegated authority.

The reporting process can be represented as:

User A

|

| Report

↓

Report Object

|

↓

Evidence / Reason

|

↓

Verification & Review

|

|— Dismissed

|— Warning

|— Restricted

|— Verified Violation

└— Escalated Review

A crucial principle is:

A report is not itself evidence of guilt.

This distinction protects the system from coordinated false reporting.

6.1 Report Aggregation

Repeated reports can be analyzed, but simple numerical accumulation should not automatically determine guilt.

For example, 100 coordinated reports should not necessarily outweigh one well-supported report.

The system should consider:

- evidence,
 - report quality,
 - independence of reporters,
 - historical reliability,
 - relationship between reporters and the reported user,
 - severity,
 - repetition,
 - and contextual relevance.
-

6.2 Appeals

A robust accountability system requires an appeal mechanism.

A user who receives a moderation or trust-related decision should be able to:

- inspect the basis of the decision,
- submit evidence,
- challenge factual errors,
- request review,
- and, where appropriate, appeal to a higher governance mechanism.

Digital deliberation research increasingly emphasizes transparent and traceable moderation and mechanisms through which users can report inappropriate content or actions.

Thus, PEPG should treat reporting as part of governance rather than as an invisible administrative function.

7. The Context Engine

The Context Engine determines the semantic and substantive context of problems, discussions, users, solutions, and projects.

For example:

Problem:

Urban River Pollution

Contexts:

Environmental Management

Water Management

Industrial Pollution

Public Health

Urban Infrastructure

Contexts can be:

- automatically suggested,
- proposed by users,
- reviewed by the community,
- accepted,

- modified,
 - or rejected.
-

7.1 Contextual User Profiles

The same engine can classify user expertise.

User #102

Civil Engineering

95%

Water Management

91%

Environmental Engineering

87%

Public Administration

61%

This does not mean that the person has four separate identities.

It means the system maintains a contextual representation of demonstrated expertise.

7.2 Dynamic Context

Context may change as discussion progresses.

A problem initially classified as:

Road Damage

may later acquire:

Drainage Failure

Construction Quality

Urban Water Management

The Context Engine should therefore maintain context history.

8. The Delegation Engine

Fig. 4 Delegation and Trust Model

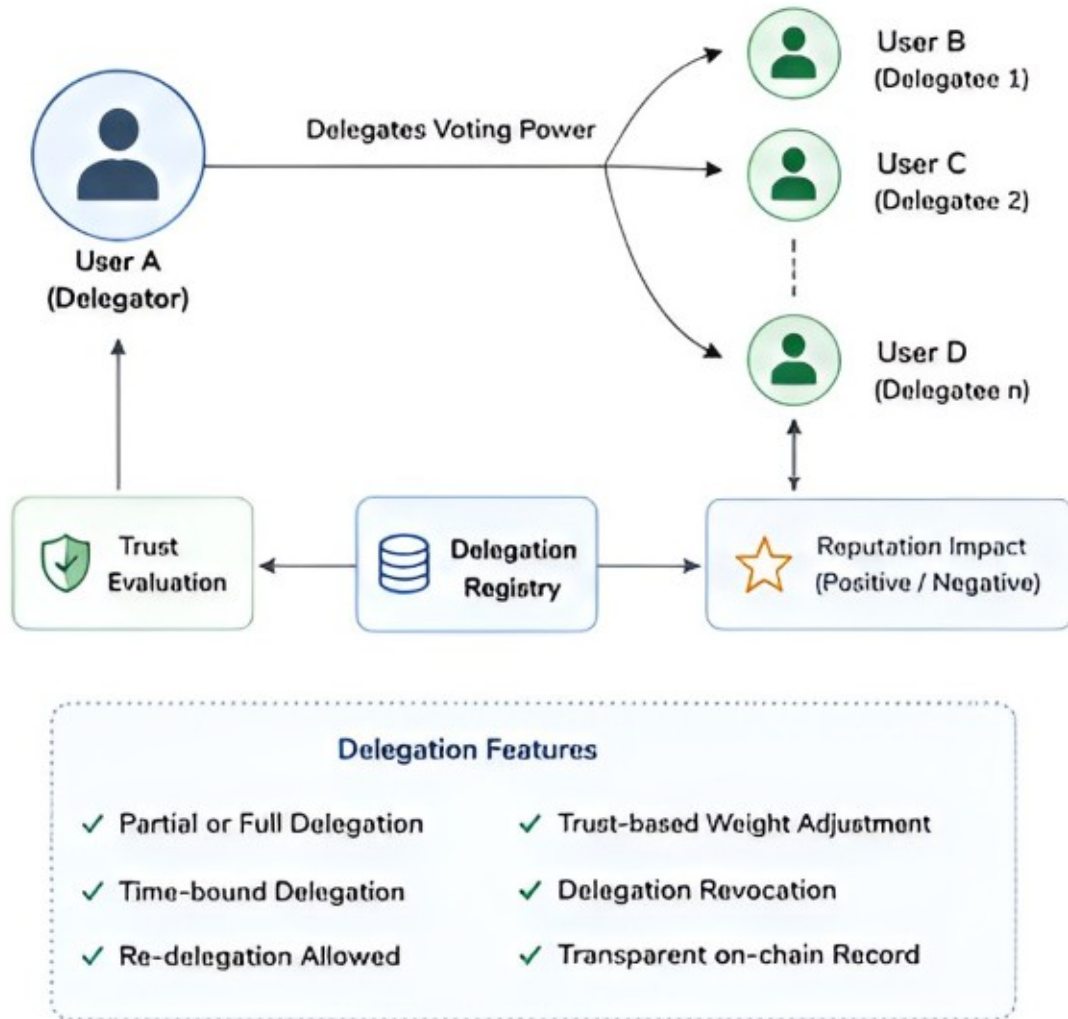


Figure 4. Delegation and trust model.

The Delegation Engine enables users to transfer voting rights.

This mechanism is related to Liquid Democracy, where voters may choose between voting directly and delegating to another participant, potentially with different representatives for different issues. Delegations can also be transitively transferred in some models.

PEPG extends this concept through explicit contextual scopes.

8.1 Delegation Scopes

A user may delegate voting rights:

Globally

“I delegate my voting rights.”

By domain

“I delegate my voting rights concerning Civil Engineering.”

By problem

“I delegate my voting rights for this problem.”

By discussion

“I delegate my voting rights for this discussion.”

By decision

“I delegate my voting rights for this specific decision.”

This produces:

Global

Domain

Problem

Discussion

Decision

as distinct delegation scopes.

8.2 Loss of Direct Voting Right

When a user delegates voting rights within a specific scope, the user’s direct voting right in that scope becomes inactive.

For example:

User A

|

└─ delegates Problem #52 → User B

User A cannot simultaneously cast a direct vote in Problem #52.

This prevents double representation.

8.3 Delegation Chains

Delegation may form a chain:

$$A \rightarrow B \rightarrow C \rightarrow D$$

If the system allows transitive delegation, D becomes the final voting node.

The literature recognizes transitive delegation as a central feature of delegable proxy systems, while also identifying technical and normative challenges such as cycles and concentration of power.

PEPG therefore requires:

- cycle detection,
 - delegation history,
 - transparent paths,
 - revocation,
 - scope resolution,
 - and concentration monitoring.
-

8.4 Delegation Graph

Rather than treating delegation as a simple database field, PEPG models it as a graph:

$$A \longrightarrow B \longrightarrow C$$

|

└────────→ D

The Voting Engine resolves the graph for each decision.

9. The Discussion and Deliberation Engine

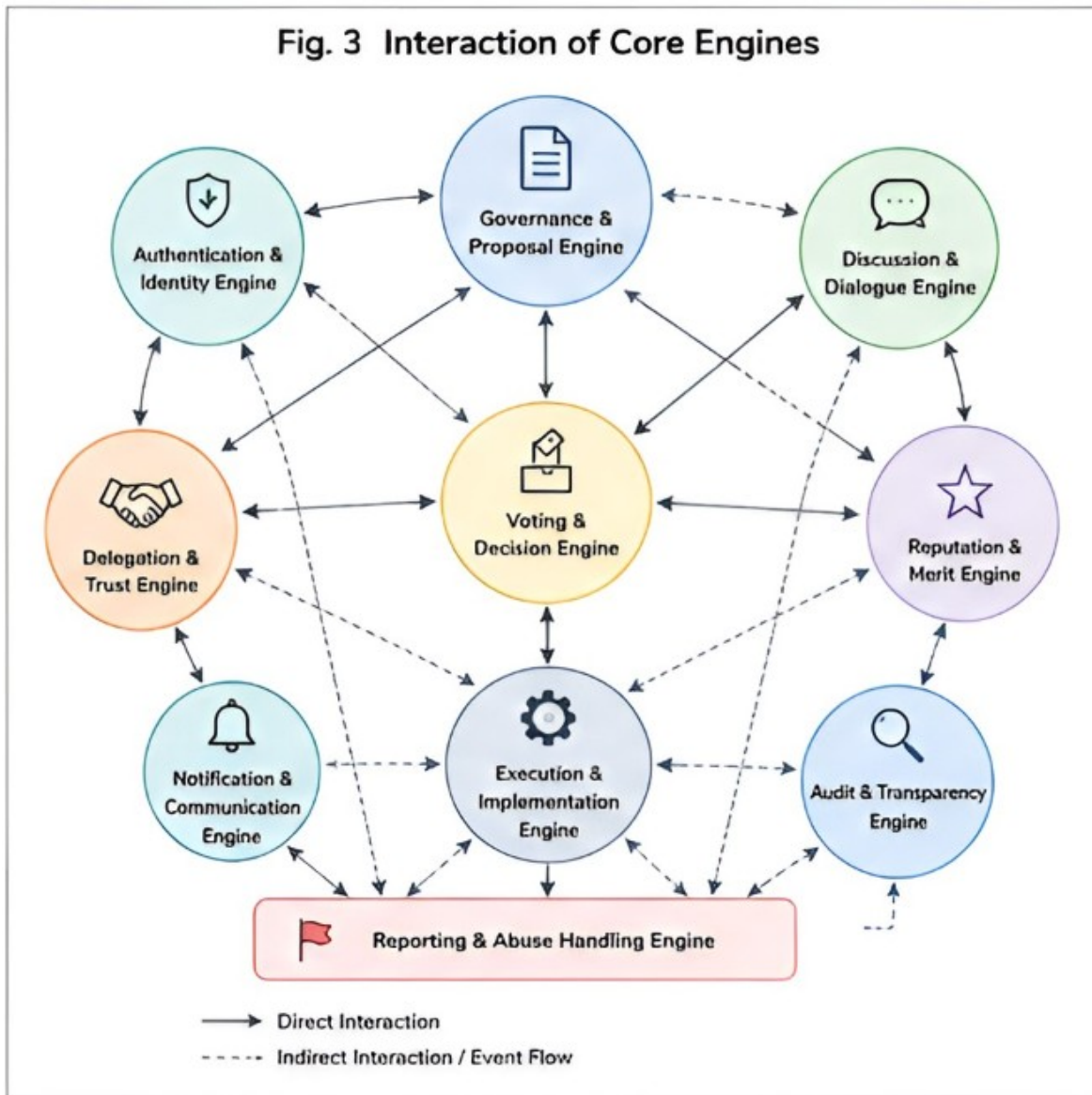


Figure 3. Interaction of the core governance engines.

The Discussion Engine is not simply a chat system.

It is a structured mechanism for converting collective communication into governance outputs.

This distinction is important because research on online deliberation emphasizes structured environments that can support equitable participation, agenda management, speaking order, and organized discussion rather than unrestricted communication alone.

10. Discussion Lifecycle

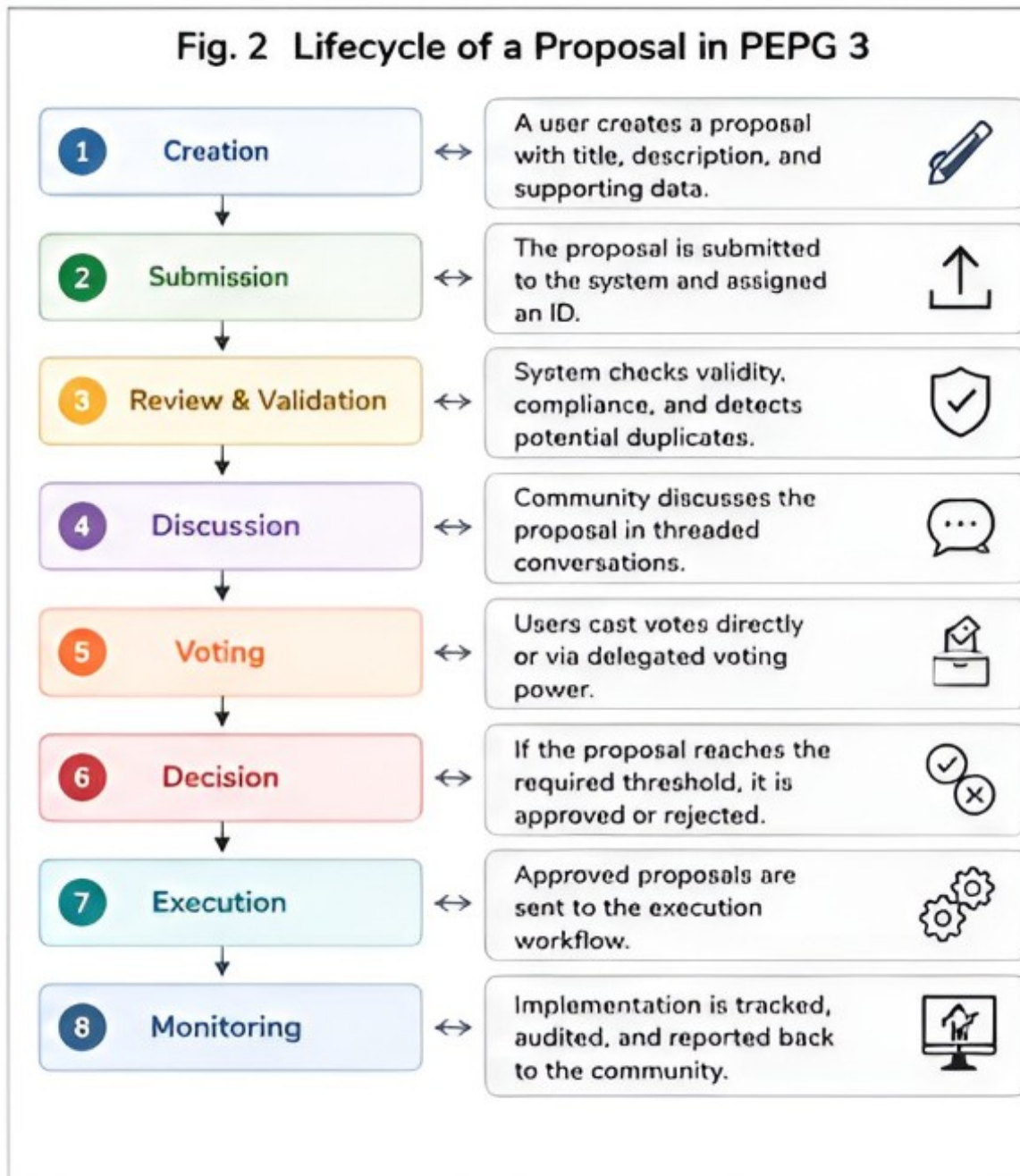


Figure 2. Lifecycle of a proposal in PEPG 3.

Every discussion follows a defined lifecycle:

Proposed

↓

Formation Voting

↓

Active

↓

Output Formation

↓

Closure Voting

↓

Closed

11. Discussion Formation

Any participant in a problem may propose a discussion.

Example:

“Should the city impose restrictions on industrial discharge?”

The proposal enters:

Discussion Formation

If at least 80% of the relevant participating voting power supports its formation, the discussion becomes active.

Eligible Voting Power = 10,000

YES = 8,300

Approval = 83%

Result:

Discussion Created

The 80% threshold is a proposed PEPG governance rule and should be understood as part of the architecture’s normative design rather than as an established democratic standard.

12. Active Discussion

An active discussion contains a structured conversation space.

Users can:

- post messages,
 - respond,
 - like,
 - dislike,
 - propose polls,
 - propose new discussions,
 - submit evidence,
 - propose solutions,
 - identify claims,
 - and request closure.
-

13. Message-Level Interaction

Each message can have:

Message

└─ Author

└─ Timestamp

└─ Content

└─ Context

└─ Evidence

└─ Likes

└─ Dislikes

└─ Replies

└─ Governance

Actions

This makes the conversation auditable.

14. Highlights

A distinctive feature of PEPG is the **Highlights** layer.

Messages that receive at least 60% positive support from the relevant population are surfaced in the Highlights section.

For example:

Discussion #204

Highlights

Message #51

82% Like

Message #77

71% Like

Message #104

64% Like

This is intended to solve a common problem of large-scale digital deliberation: valuable information becomes buried inside thousands of messages.

However, a Like should not be interpreted as factual verification.

Therefore:

Like $\neq$ Truth

Like $\neq$ Official Verification

Like $\neq$ Expert Validation

It is an expression of collective approval, relevance, usefulness, agreement, or resonance according to the platform's defined interaction semantics.

15. New Discussions Inside Existing Discussions

A discussion may generate a new question.

For example:

Main Discussion:

Industrial Pollution

↓

New Question:

Which factory is responsible?

↓

Proposed Sub-discussion:

Assessment of Factory X

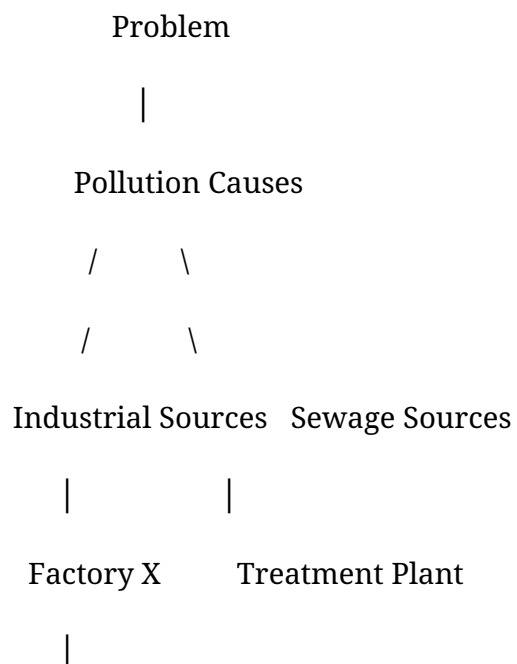
If the required threshold is reached, the new discussion becomes active.

This creates a **Discussion Graph**.

16. Discussion Map

The Discussion Map visually represents relationships among discussions.

For example:



Compliance

The structure may resemble a tree, but architecturally it is more accurately represented as a graph because discussions may have multiple relationships.

Possible relations include:

- parent,
 - child,
 - related,
 - dependent,
 - contradictory,
 - extension,
 - duplicate,
 - evidence for,
 - consequence of.
-

17. Polls and Decisions Inside Discussions

Users can create polls within discussions.

For example:

Which intervention should be prioritized?

A — Industrial monitoring

B — New treatment facility

C — Both

The result becomes part of the discussion record.

This allows the system to distinguish between:

- conversational messages,
 - opinion polls,
 - formal proposals,
 - and final decisions.
-

18. Discussion Outputs

Every discussion should produce an identifiable output.

An output may contain:

Finding

Evidence

Arguments

Areas of disagreement

Proposals

Recommended solution

Unresolved questions

For example:

Discussion Output

Finding:

Industrial discharge is a major contributor.

Evidence:

Three verified reports.

Recommendation:

Implement continuous industrial monitoring.

Unresolved:

Funding mechanism.

This prevents discussion from becoming an infinite conversation.

19. Closing a Discussion

Any participant may propose closure.

The proposal enters a closure vote.

If at least 80% of the relevant voting power supports closure:

Active Discussion



80% Closure

Approval



Closed Discussion

A closed discussion remains permanently accessible as historical governance data.

20. Discussion Closure Is Not Problem Closure

This distinction is fundamental:

Discussion Closure $\neq$ Problem Closure

A problem can contain multiple discussions.

Some may close while others remain active.

The problem itself only becomes eligible for closure after its broader solution and execution lifecycle has been completed.

21. The Solution Engine

Discussion outputs can become formal solution proposals.

A solution is classified into two primary categories:

Solution



├— Theoretical Solution



└— Practical Solution

21.1 Theoretical Solution

A theoretical solution is considered sufficient when the problem can be resolved through knowledge, explanation, clarification, policy interpretation, or another non-execution outcome.

If the required 80% approval threshold is reached:

Theoretical Solution

↓

80% Approval

↓

Problem Closure Process

21.2 Practical Solution

A practical solution requires implementation.

For example:

Construct a new water treatment facility.

If the required approval is achieved:

Practical Solution

↓

80% Approval

↓

Execution Phase

↓

Project Creation

Again, the 80% threshold is a proposed PEPG design parameter rather than a universal democratic principle.

22. The Voting Engine

The Voting Engine resolves formal decisions.

It receives information from:

- Trust Engine,
- Context Engine,
- Delegation Engine,
- Discussion Engine,
- and the relevant governance scope.

The basic process is:

Decision



Eligible Participants



Delegation Resolution



Voting Power Calculation



Vote Collection



Result



Audit Record

23. Voting Power

Voting power is not necessarily identical to the number of accounts.

If:

A = 1 vote

B = 1 vote

C = 1 vote

and:

$A \rightarrow B$

$C \rightarrow B$

then B may exercise:

Own vote = 1

Delegated votes = 2

Total voting power = 3

This is conceptually consistent with proxy-based liquid voting models.

However, PEPG must explicitly address concentration risk. Research has shown that delegation can create highly concentrated influence and can sometimes reduce informational diversity.

Therefore, PEPG should expose delegation concentration transparently rather than hiding it.

24. The Execution and Project Governance Engine

Once a practical solution is approved, PEPG moves from deliberation to execution.

The practical solution becomes one or more projects.

Problem

↓

Practical Solution

↓

Execution Decision

↓

Project

25. Project Structure

Each project may contain:

Project

|

├─ Project
Manager

├─ Execution
Team

├─ Supervisors

├─ Suppliers

├─ Requirements

├─ Budget

├─ Timeline

├─ Milestones

├─ Contracts

├─ Challenges

├─ Progress
Reports

├─ Evidence

└─ Final
Evaluation

26. Sub-Projects

A project may contain multiple sub-projects.

For example:

Urban Water Project

|

|— Engineering
Design

|— Land Preparation

|— Construction

|— Equipment
Procurement

|— Environmental
Assessment

|— Final
Inspection

Each sub-project may possess its own:

- manager,
 - team,
 - budget,
 - schedule,
 - requirements,
 - reports,
 - and verification.
-

27. Project Roles

Project

Manager

Responsible for:

- coordination,

- planning,
- progress reporting,
- challenge identification,
- requirements,
- and project status.

Execution

Team

Responsible for implementing the approved plan.

Supervisors

Independently assess:

- progress,
- quality,
- compliance,
- evidence,
- and completion.

Suppliers

Provide:

- materials,
 - equipment,
 - services,
 - or other required resources.
-

28. Project Selection and Trust

The selection of managers, suppliers, and supervisors can use verified contextual profiles.

For example:

Candidate Manager

Civil Engineering: 96% verified

Project Management: 91%

Previous Projects: 14

Relevant Reputation: 88%

The system therefore does not merely ask:

Who is available?

It can ask:

Who is verified and contextually qualified for this project?

29. Requirements

Every project maintains a requirements structure.

Requirements can include:

- technical specifications,
- human resources,
- equipment,
- materials,
- budget,
- legal requirements,
- environmental conditions,
- delivery milestones.

A new requirement may arise during execution.

Such a requirement should not silently modify the original decision.

Instead:

New Requirement

↓

Requirement Proposal

↓

Review

↓

Discussion / Vote if required

↓

Approval



Project Update

This preserves governance continuity.

30. Contracts and Blockchain

When the relevant participants reach final agreement, contractual information may be recorded on blockchain infrastructure.

Participants may include:

- community representatives,
- project manager,
- executor,
- supervisors,
- suppliers,
- and other contractual parties.

The purpose of blockchain is not to make every piece of project information immutable.

Rather, it can provide a tamper-evident record for selected governance commitments.

A conceptual contract record might include:

Contract

└─ Contract ID

└─ Project ID

└─ Parties

└─ Scope

└─ Budget

└─ Timeline

└─ Responsibilities

└─ Conditions

└─ Amendments

└─ Digital
Signatures

└─ Blockchain
Reference

Blockchain should therefore function as an **audit and commitment layer**, not as a substitute for governance.

31. Project Progress

The Project Manager can continuously update:

- percentage completed,
- milestones,
- delays,
- challenges,
- resource requirements,
- financial status,
- and evidence.

Example:

Project Progress

Physical Progress: 47%

Time Elapsed: 51%

Budget Used: 43%

Open Challenges: 3

New Requirements: 2

32. Challenges

A project challenge becomes a governance object.

For example:

Construction materials have been delayed by 30 days.

The manager records:

Challenge

└─ Description

└─ Impact

└─ Evidence

└─ Proposed
Response

└─ Required
Decision

If a collective decision is required, the challenge can return to the Discussion and Voting Engines.

Thus, execution remains connected to deliberation.

33. Independent Supervision

A central principle is that project reporting and project verification should remain separate.

The manager may report:

80% complete.

The supervisor independently reports:

73% verified.

Both records remain available.

This creates an important distinction:

Self-Reported Progress

≠

Verified Progress

34. Project Completion

When the manager declares the project complete:

Manager Completion Declaration

↓

Supervisor Inspection

↓

Supervisor Verification

↓

Final Project Report

↓

Community Review

↓

Problem Closure Vote

The project therefore cannot unilaterally declare the problem solved.

35. Problem Closure

Problem closure requires the conditions defined by the PEPG governance rules.

For a practical problem, for example:

1. Project completed.
2. Required supervisors approve completion.
3. Final reports are available.
4. Relevant evidence is available.
5. Community performs final evaluation.
6. Required final vote is achieved.

Then:

Problem

↓

Execution Completed



Verified



Community Approved



Closed

36. The Accountability and Reporting Engine

Fig. 5 Reporting and Abuse Handling Workflow

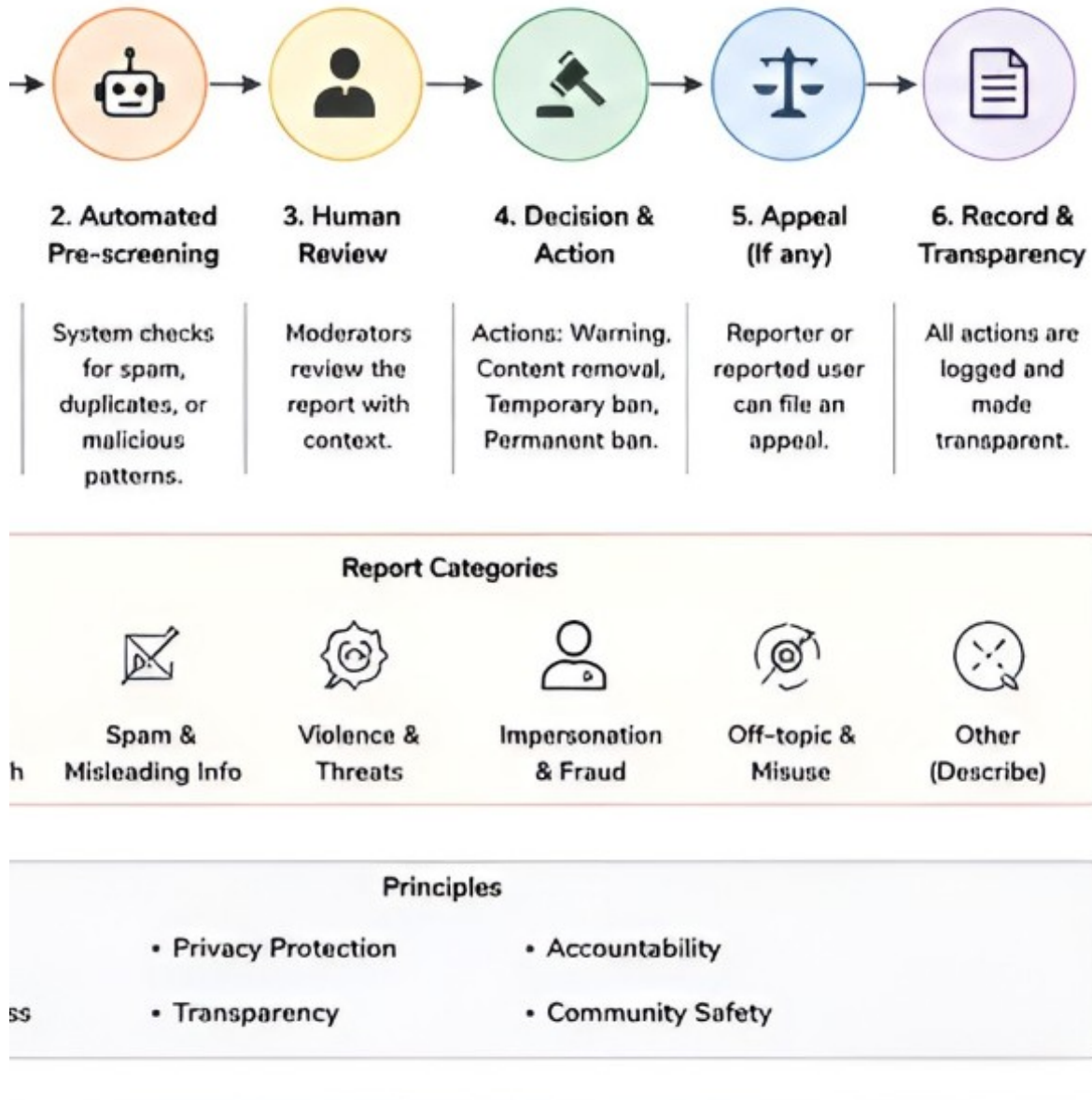


Figure 5. Reporting and abuse-handling workflow.

All major PEPG actions produce records.

This includes:

- identity verification,
- user reports,
- delegation,
- votes,

- discussions,
- proposals,
- project updates,
- supervisor reports,
- contracts,
- amendments,
- and closure.

The Reporting Engine provides a historical view of the problem.

For example:

Problem #1024

Timeline

Day 1

Problem created

Day 3

Context approved

Day 5

Discussion #1 created

Day 14

Discussion #1 closed

Day 20

Practical solution approved

Day 25

Project created

Day 27

Contract recorded

Day 60

Progress: 40%

Day 90

Supervisor challenge

Day 120

Project completed

Day 125

Final verification

Day 130

Problem closed

This creates a **Governance Audit Trail**.

37. The Reporting Engine and Evidence

Reports should support evidence.

For example:

Report

└─ Author

└─ Role

└─ Timestamp

└─ Statement

└─ Evidence

└─ Related

Project

└─ Related

Requirement

└─ Verification Status

Evidence can include:

- documents,
 - photographs,
 - measurements,
 - certificates,
 - datasets,
 - inspection records,
 - and external institutional references.
-

38. Governance Integrity

Fig. 6 Data and Trust Flow in PEPG 3

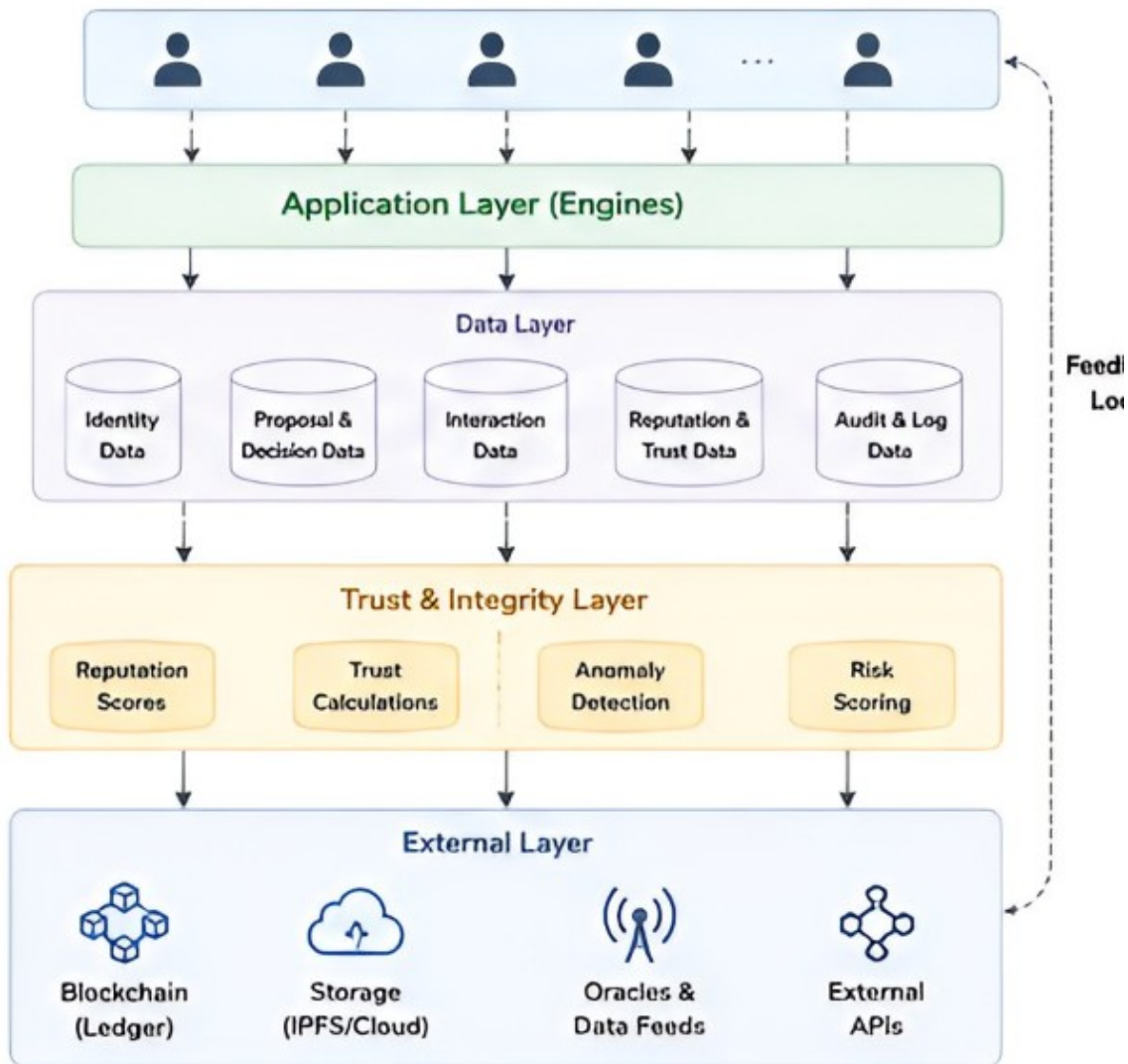


Figure 6. Data and trust flow in PEPG 3.

PEPG should maintain the distinction between:

Claim

Evidence

Verification

Community Support

Decision

Execution

Outcome

These are not interchangeable.

For example:

Claim:

The bridge is 80% complete.

Evidence:

Construction records.

Verification:

Supervisor confirms 76%.

Community Reaction:

72% positive.

Decision:

Continue project.

Outcome:

Final completion after inspection.

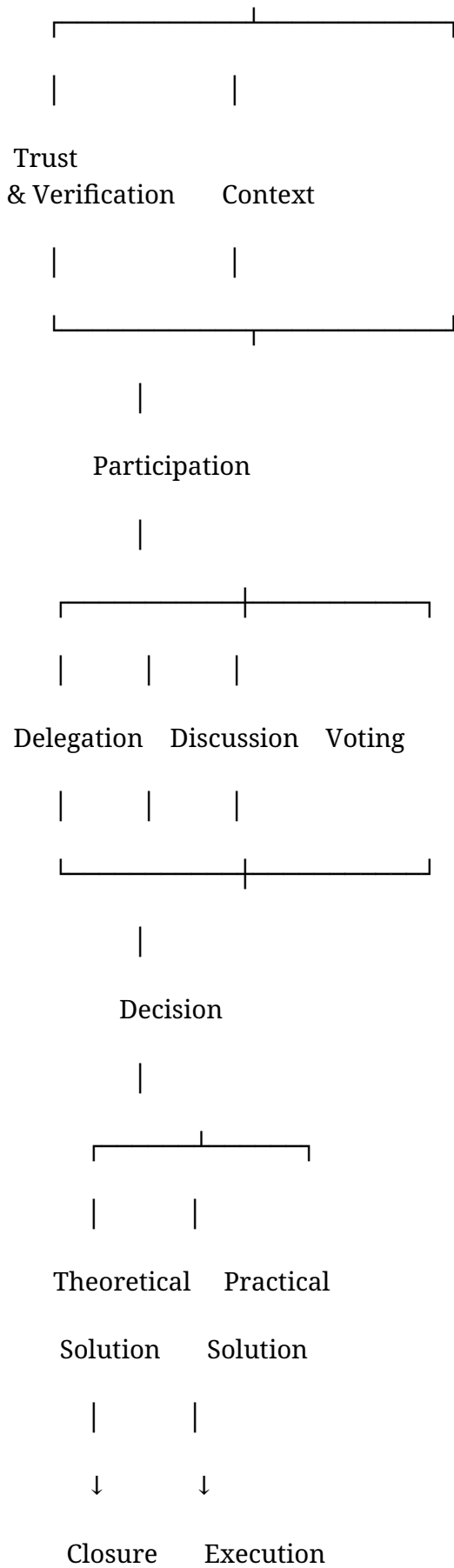
This layered structure is central to accountable digital governance.

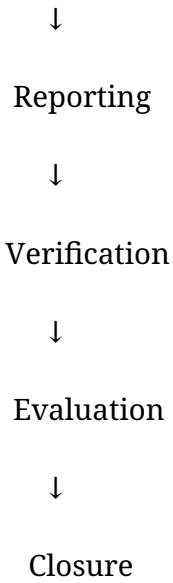
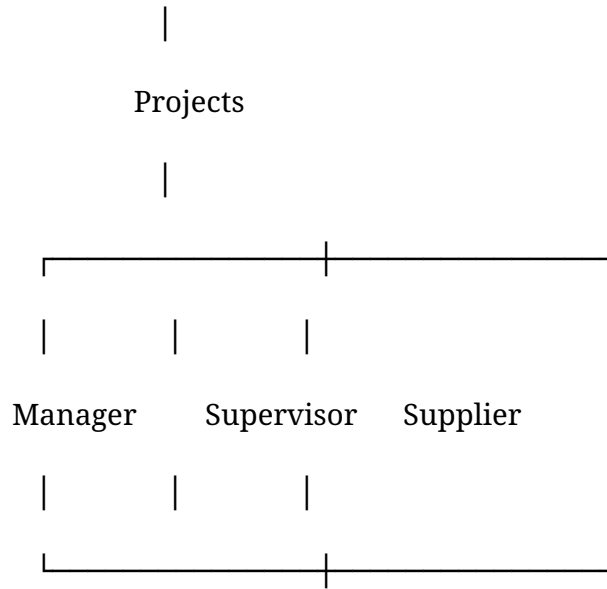
39. Interaction Among the Engines

The engines form an integrated architecture.

PEPG

|

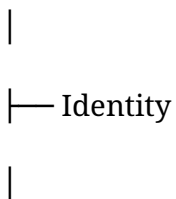




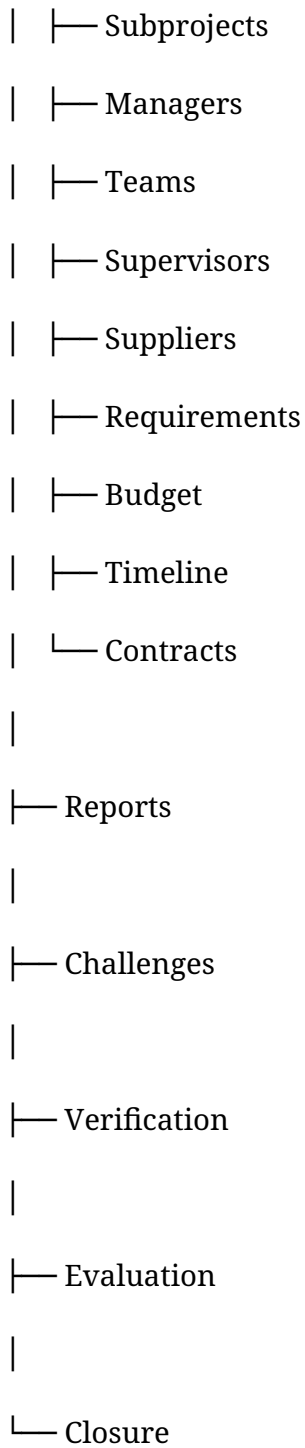
40. The Problem as a Governance Container

The complete architecture can be understood through the following object:

PROBLEM



└─ Context	
└─ Participants	
└─ Trust Records	
└─ Delegations	
└─ Discussions	
└─ Active	
└─ Proposed	
└─ Closed	
└─ Discussion Map	
└─ Evidence	
└─ Proposals	
└─ Solutions	
└─ Votes	
└─ Projects	



This is arguably the most important architectural concept of PEPG.

The problem becomes a **persistent governance object**.

41. User Lifecycle

The user also possesses a governance lifecycle.

Registration



Identity Verification



Profile Verification



Contextual Expertise



Participation



Delegation



Deliberation



Voting



Project Participation



Performance



Reputation



Future Representation

Thus, actions in one problem can contribute to the contextual reputation of a participant in future problems, subject to explicit privacy, fairness, and governance rules.

42. Collective Intelligence

PEPG attempts to combine several forms of collective intelligence:

Distributed

Knowledge

Different users possess different information.

Expertise

Some users possess verified expertise.

Local

Knowledge

Some participants possess direct experience with the problem.

Delegated

Knowledge

Users can voluntarily rely on participants they trust.

Deliberative

Knowledge

Participants exchange arguments and evidence.

Operational

Knowledge

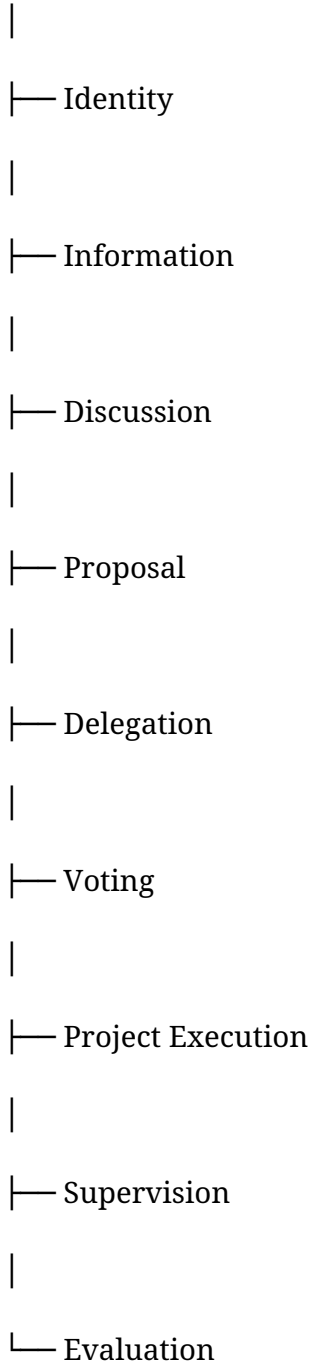
Project participants contribute implementation knowledge.

The system therefore does not reduce collective intelligence to a simple vote count.

43. A Multi-Layer Model of Participation

PEPG can consequently be understood as a sequence of increasingly structured participation:

Participation



This is a significant conceptual difference from systems in which participation begins and ends with voting.

44. Theoretical Positioning

PEPG draws conceptually from several research traditions but should not be reduced to any one of them.

Liquid

Democracy

PEPG shares with Liquid Democracy:

- voluntary delegation,
- proxy representation,
- direct voting,
- contextual or issue-specific representation,
- and potentially transitive delegation.

The academic literature identifies voluntary delegation and proxy voting as core characteristics of Liquid Democracy.

However, PEPG extends delegation into a broader governance architecture.

Deliberative

Democracy

PEPG also draws on deliberative principles by giving participants mechanisms to:

- exchange reasons,
- examine evidence,
- challenge assumptions,
- form sub-discussions,
- formulate proposals,
- and reach structured outputs.

Existing online deliberation platforms demonstrate the value of structured participation, speaking management, agendas, and mechanisms designed to support more equitable conversation.

Platform

Governance

PEPG also incorporates mechanisms associated with platform governance:

- moderation,
- reporting,
- accountability,
- transparency,
- traceability,
- user participation,
- and appeals.

Recent scholarship emphasizes that moderation increasingly involves interactions between algorithmic systems and human decision-makers, demonstrating why governance mechanisms cannot be treated as purely technical components.

45. Risks and Design Challenges

A serious academic treatment of PEPG must also acknowledge its risks.

45.1 Delegation Concentration

Delegation can accumulate around highly visible participants.

Research has demonstrated that delegated systems can experience substantial concentration of voting power.

PEPG therefore needs:

- transparent delegation graphs,
 - concentration indicators,
 - contextual delegation,
 - revocation,
 - and potentially safeguards against excessive accumulation.
-

45.2 Overdelegation

Delegation can reduce the diversity of independent information entering a decision.

Experimental research has found circumstances in which delegation performs worse than direct voting or abstention, particularly when participants overestimate the knowledge of delegates.

Therefore:

Delegation should be an option, not an assumption.

45.3 False Verification

Community verification can be manipulated through coordinated confirmations.

Therefore, five users confirming a claim should not necessarily be treated as equivalent to an official institutional verification.

The system should preserve the provenance of every verification.

45.4 False Reporting

A reporting system can itself be abused.

Therefore:

Report

≠

Violation

Reports require evidence and procedural review.

45.5 Popularity versus Truth

A highly liked message may be incorrect.

Therefore:

Popularity

≠

Truth

This is why PEPG requires separate:

- social signals,
 - verification,
 - evidence,
 - and institutional confirmation.
-

45.6 Governance Capture

Highly active participants may accumulate:

- reputation,
- delegation,
- visibility,
- and influence.

PEPG therefore requires mechanisms to make concentrations of power visible.

46. Transparency as a Core Property

A user should be able to understand not only the result but also its provenance.

For example:

Decision:

Approve Project X

Result:

82% YES

Voting Power:

10,000

Direct Votes:

4,200

Delegated Votes:

5,800

Discussion:

Discussion #102

Solution:

Solution #33

Evidence:

14 verified
documents

Delegation Graph:

Available

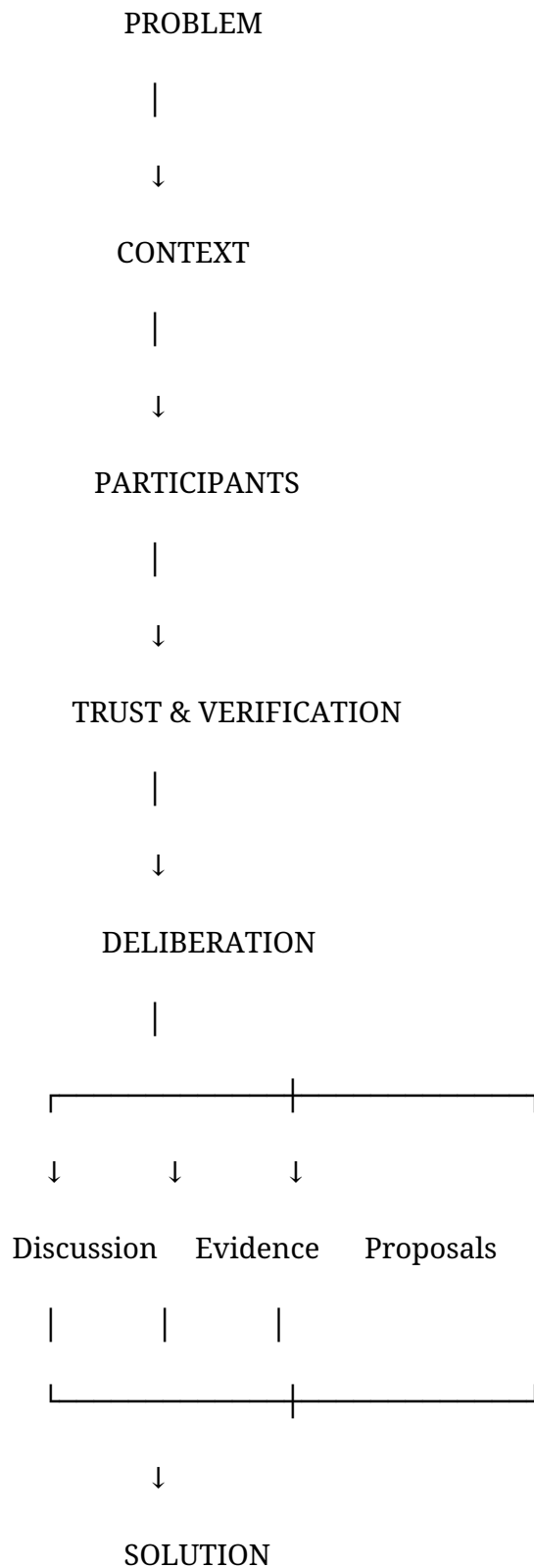
Final Decision:

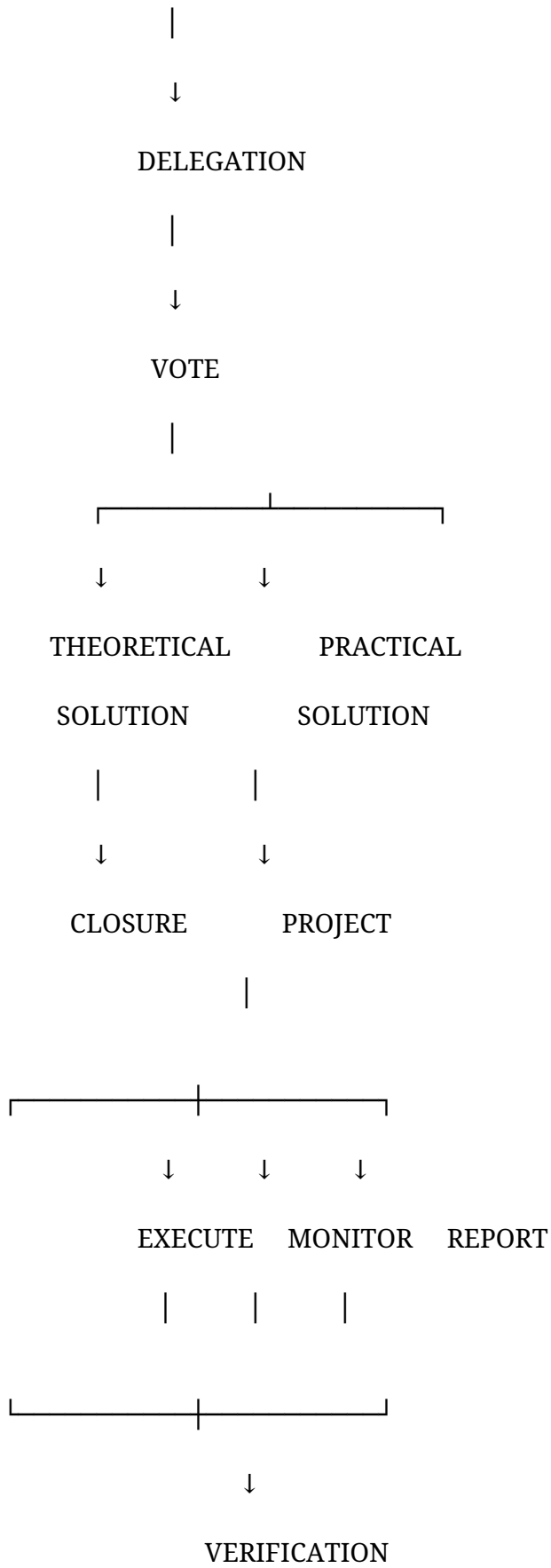
Approved

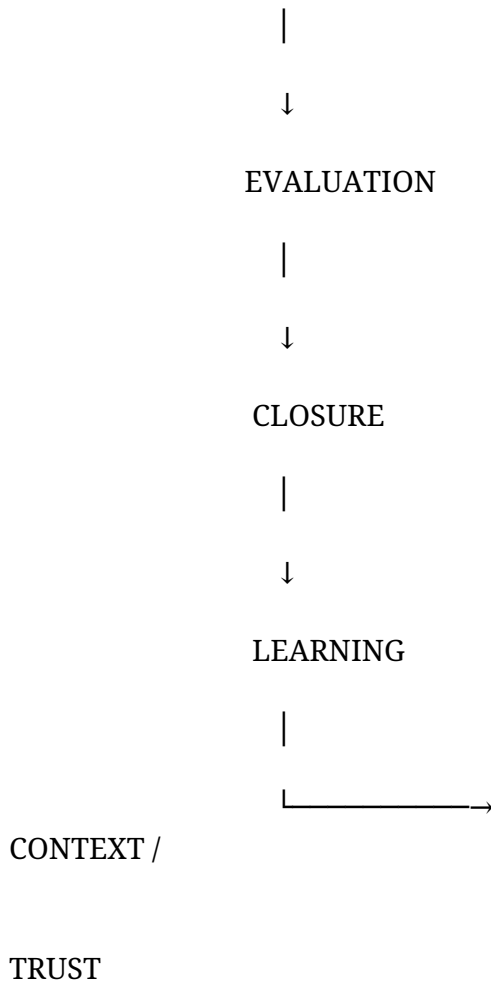
This produces **explainable governance**.

47. The PEPG Governance Cycle

The complete architecture can now be summarized:







48. PEPG as a Governance Operating System

The architecture ultimately suggests that PEPG should not be conceptualized as:

a voting application,

or:

a social network,

or:

a project-management platform.

Rather, it is better conceptualized as a:

Its purpose is to coordinate the complete lifecycle of collective problems.

The operating system metaphor is useful because the individual engines perform specialized functions while sharing common governance primitives.

These primitives include:

- identity,
 - trust,
 - context,
 - participation,
 - delegation,
 - evidence,
 - decision,
 - accountability,
 - and execution.
-

49. Proposed Core Governance Engines

The complete PEPG architecture can therefore be represented by the following engines:

1. Identity

& Profile Engine

Manages user identity and profile data.

2. Trust

& Verification Engine

Validates user claims and establishes field-level confidence.

3. Context

Engine

Classifies users, problems, discussions, solutions, and projects.

4. Delegation

Engine

Manages contextual transfer of voting rights.

5. Discussion

& Deliberation Engine

Creates and manages structured collective discussion.

6. Voting

Engine

Calculates eligible voting power and resolves formal decisions.

7. Solution

Engine

Converts deliberative outputs into theoretical or practical solutions.

8. Execution

& Project Governance Engine

Transforms approved practical solutions into governed projects.

9. Contract

Engine

Records contractual commitments and selected immutable audit references.

10. Monitoring

& Reporting Engine

Maintains project progress and governance records.

11. Verification

& Inspection Engine

Separates claims from independently verified results.

12. Accountability

& Reporting Engine

Manages user reports, violations, appeals, and governance accountability.

13. Reputation

Engine

Builds contextual records of demonstrated performance and reliability.

14. Evaluation

& Closure Engine

Determines whether a problem has been satisfactorily resolved.

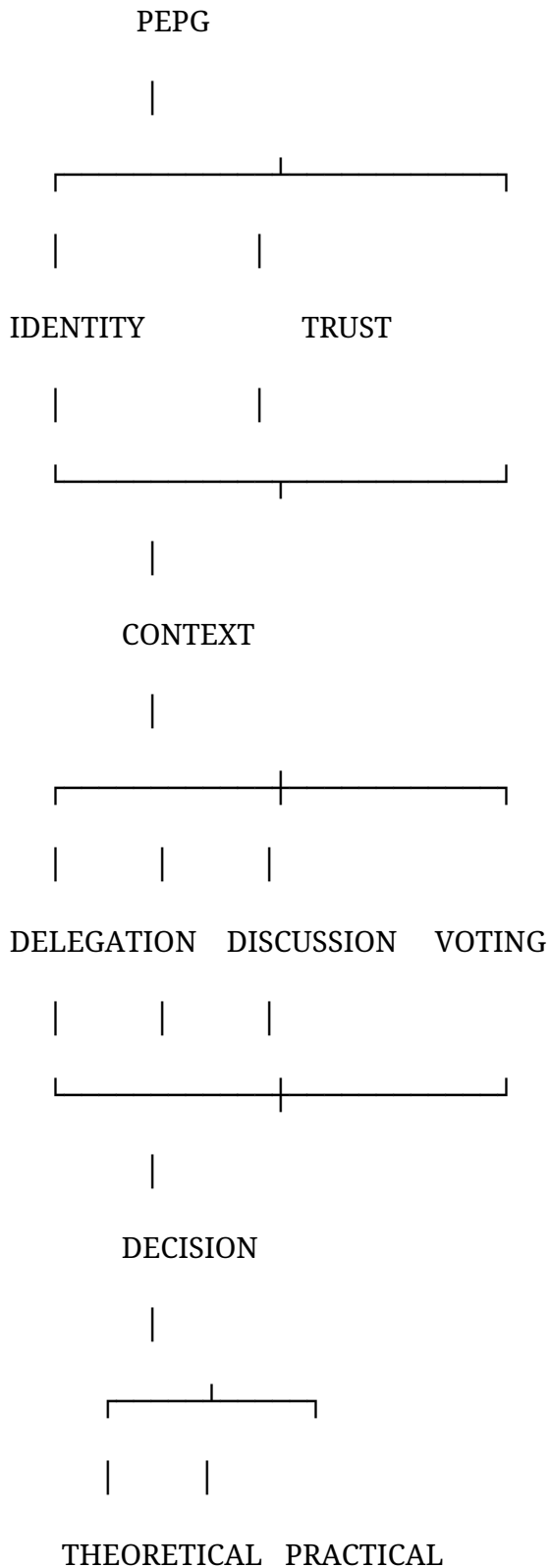
15. Knowledge

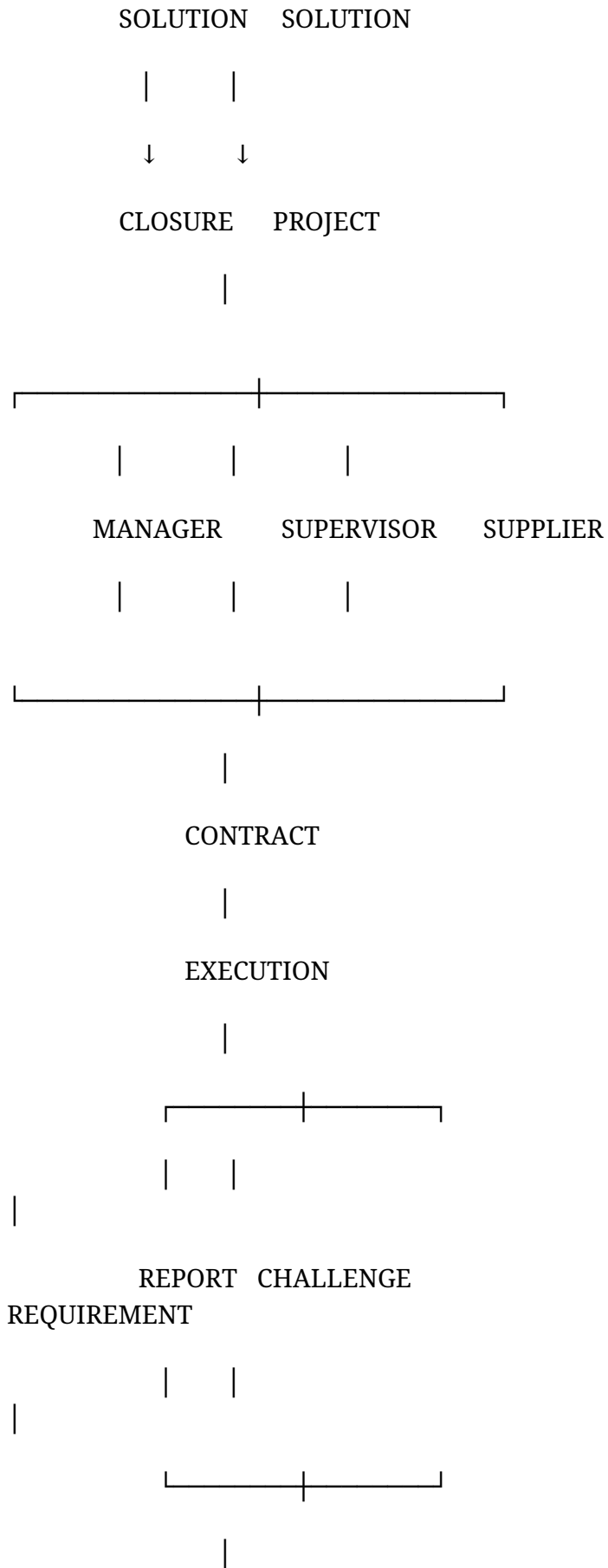
& Learning Layer

Feeds outcomes and lessons back into future contextual classification and governance.

50. Final Conceptual Model

The complete PEPG architecture can therefore be expressed as:





VERIFICATION

|

EVALUATION

|

CLOSURE

|

LEARNING

|

↓

FUTURE CONTEXT

51. Conclusion

PEPG proposes a different way of thinking about digital governance.

The fundamental unit is not the election, the vote, the user, the discussion, or the project in isolation.

It is the **problem and its complete governance lifecycle**.

A participant first establishes an identity and a verifiable profile. Claims concerning identity, education, expertise, professional activity, and other attributes can receive contextual verification. Users can then participate directly or delegate their voting rights to other participants according to the scope of a problem, discussion, domain, or decision.

Within each problem, participants can create discussions. Proposed discussions become active when the required participation threshold is reached. Active discussions provide structured communication spaces in which users can exchange arguments, provide evidence, react to messages, create polls, propose new discussions, formulate solutions, and identify important contributions. Discussions produce structured outputs rather than remaining indefinite conversations.

These outputs can become theoretical or practical solutions. A theoretical solution may lead directly to problem closure, while an approved practical solution initiates an execution process. The execution process creates projects and potentially sub-projects, each with managers, teams, supervisors, suppliers, requirements, budgets, schedules, contracts, reports, challenges, and monitoring mechanisms.

The project is not considered successful merely because its manager declares completion. Independent supervision and final community evaluation remain part of the governance lifecycle. Only after the required verification and final decision mechanisms are satisfied can the original problem be formally closed.

At the same time, PEPG introduces accountability mechanisms throughout the system. Users can report other participants, but a report is treated as an allegation rather than a verdict. Evidence, procedural review, transparency, and appeals are therefore essential components of the trust architecture.

The result is a system in which:

****identity supports trust;**
 trust supports participation;
 context determines relevance;
 delegation enables flexible representation;
 deliberation produces collective intelligence;
 voting produces decisions;
 decisions create projects;
 projects produce measurable outcomes;
 verification establishes accountability;
 and outcomes generate knowledge for future governance.**

The resulting model can be summarized as:

PEPG transforms digital participation from an isolated act of voting into a continuous, contextual, trust-aware and executable governance process.

Its most distinctive conceptual proposition is therefore not simply that citizens can vote or delegate their votes. Rather, it is that **representation, deliberation, decision-making, execution, verification, and accountability can be integrated around the lifecycle of a collective problem.**

This positions PEPG at the intersection of liquid democracy, deliberative democracy, collective intelligence, digital platform governance, trust systems, and project governance, while proposing an architectural synthesis that goes beyond any one of these traditions.

The resulting paradigm may be described as:

****Contextual,**

Trust-Centered, Problem-Oriented Participatory Governance**

or, more concisely:

****Problem-Oriented**

Digital Governance**

In this model, democracy is no longer conceptualized solely as the periodic aggregation of preferences. It becomes a continuous institutional process through which people establish trustworthy identities, exchange knowledge, deliberate, delegate authority, make decisions, execute collective action, verify outcomes, and learn from the consequences of previous decisions.

That is the central architectural proposition of PEPG.